

Decrypt Kerberos/NTLM “encrypted stub data” in Wireshark

 medium.com/tenable-techblog/decrypt-encrypted-stub-data-in-wireshark-deb132c076e7

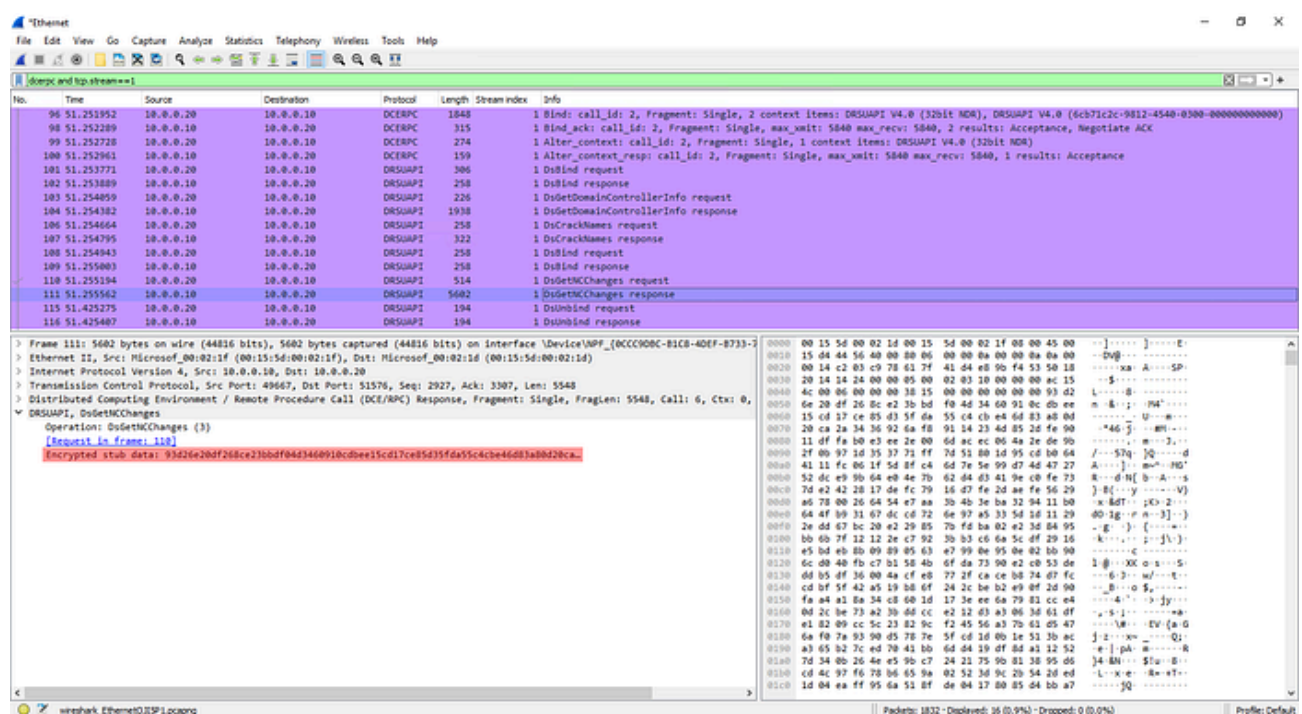
Clément Notin [Tenable]

September 28, 2022

Learn how Tenable finds new vulnerabilities and writes the software to help you find them

I often use Wireshark to analyze Windows and Active Directory network protocols, especially those juicy RPC 😊 But I’m often interrupted in my enthusiasm by the payload dissected as “encrypted stub data”:

Press enter or click to view image in full size



Can we decrypt this “encrypted stub data?” 🤔

The answer is: yes, we can! 💪 We can also decrypt Kerberos exchanges, TGTs and service tickets, etc! And same for NTLM/NTLMSSP, as I will show you near the end. Read along to learn how to decrypt DCE/RPC in Wireshark.

Wait, is that magic?

Wireshark is very powerful, as we know, but how can it decrypt data? Actually there’s no magic required because we’ll just give it the keys it needs.

The key depends on the chosen algorithm (RC4, AES128, AES256...) during the Kerberos exchange, and they derive from the password (this is simplified but you didn’t come here to read the Kerberos RFC, right? 🤪).

My preferred method to get the Kerberos keys is to use [mimikatz DCSync](#) for the target user:

```
mimikatz(commandline) # lsadump::dcsync /user:lab\krbtgt
[DC] 'lab.lan' will be the domain
[DC] 'DC.lab.lan' will be the DC server
[DC] 'lab\krbtgt' will be the user account

Object RDN          : krbtgt

** SAM ACCOUNT **

SAM Username       : krbtgt
Account Type       : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration : 
Password last change : 12/17/2020 8:15:03 PM
Object Security ID  : S-1-5-21-1891480667-311803191-3341389180-502
Object Relative ID  : 502

Credentials:
Hash NTLM: fbb44148f5f9da7490fa85fef181d88c
ntlm- 0: fbb44148f5f9da7490fa85fef181d88c
lm - 0: bb5d513ed5d9fc39fdce29920f24f58f

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
  Random Value : b40a3a4628d746d1ec1790d0471f2676

* Primary:Kerberos-Newer-Keys *
  Default Salt : LAB.LANkrbtgt
  Default Iterations : 4096
  Credentials
    aes256_hmac      (4096) : aba27ba3370e53adad8907ac6fbf3e5915f287c09a997379f8565b6f130f4d40
    aes128_hmac      (4096) : fb92e4dc4f41c53e2e84e261f10c9291
    des_cbc_md5      (4096) : a4d30d98bfc194df
```

You'll directly notice the AES256, AES128, and DES keys at the bottom, but what about the RC4 key? As you may have guessed, it's simply the NT hash 😊

Just remember that modern Windows environments will likely use AES256 so that's what we'll target.

Keep tabs on the keys

Kerberos keys are commonly stored in "keytab" files, especially on Linux systems. By the way, if you find a keytab during a pentest, don't forget to extract its keys because you'll be able to create a silver ticket against the service, as I once did (see below ) , or access other services with this identity.

So it's no surprise that Wireshark expects its keys in a keytab too. It's a binary format which can contain several keys, for different encryption algorithms, and potentially for different users.

Wireshark wiki describes [how to create the keytab file](#), using various tools like *ktutil*. But the one I found the most convenient is [keytab.py](#), by Dirk-jan [@_dirkjan](#) Mollema, who wrote it to decrypt Kerberos in his research on [Active Directory forest trusts](#). I especially

like that it doesn't ask for the cleartext password, just the raw keys, contrary to most other tools.

First, download [keytab.py](#) (you don't even need the entire repo). Additionally, [install impacket](#) if you have not already done so.

Then, open the script and edit lines 112 to 118 and add all the keys you have (in hexadecimal format) with the number corresponding to their type. For example, as we said, most of the time AES256 is used, corresponding to type 18.

The more keys you have, the better 🎉 If you are hesitant, you can even include the RC4 and AES256 keys for the same user. As Dirk-jan comments in the code, you can include the "krbtgt" key, "user" keys (belonging to the client user), "service" keys (belonging to the service user), and even "trust" keys (if you want to decrypt referral tickets in inter-realm Kerberos authentications). You can also add "computer account" keys to decrypt machines' Kerberos communications (machine accounts in AD are users after all! Just don't forget the dollar at the end when requesting their keys with DCSync). You don't need to worry about the corresponding username or domain name in the keytab; it doesn't matter for Wireshark.

Finally, run the script and pass the output filename as argument:

```
$ python keytab.py keytab.kt
```

Back to Wireshark

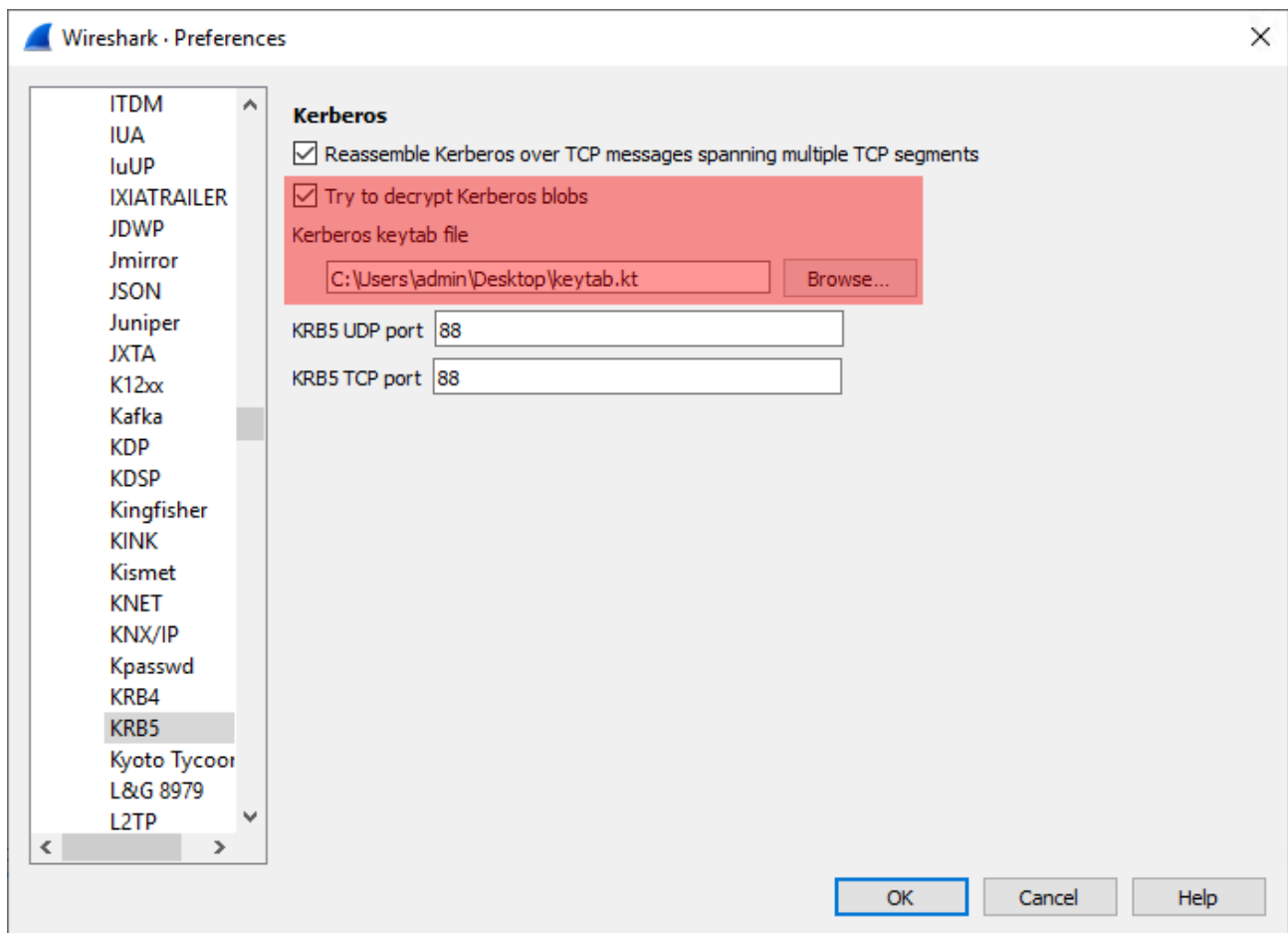
Configuration

Now that you have the keytab, open the Wireshark Preferences window, and under Protocols, look for "KRB5".

Join Medium for free to get updates from this writer.

Remember me for faster sign in

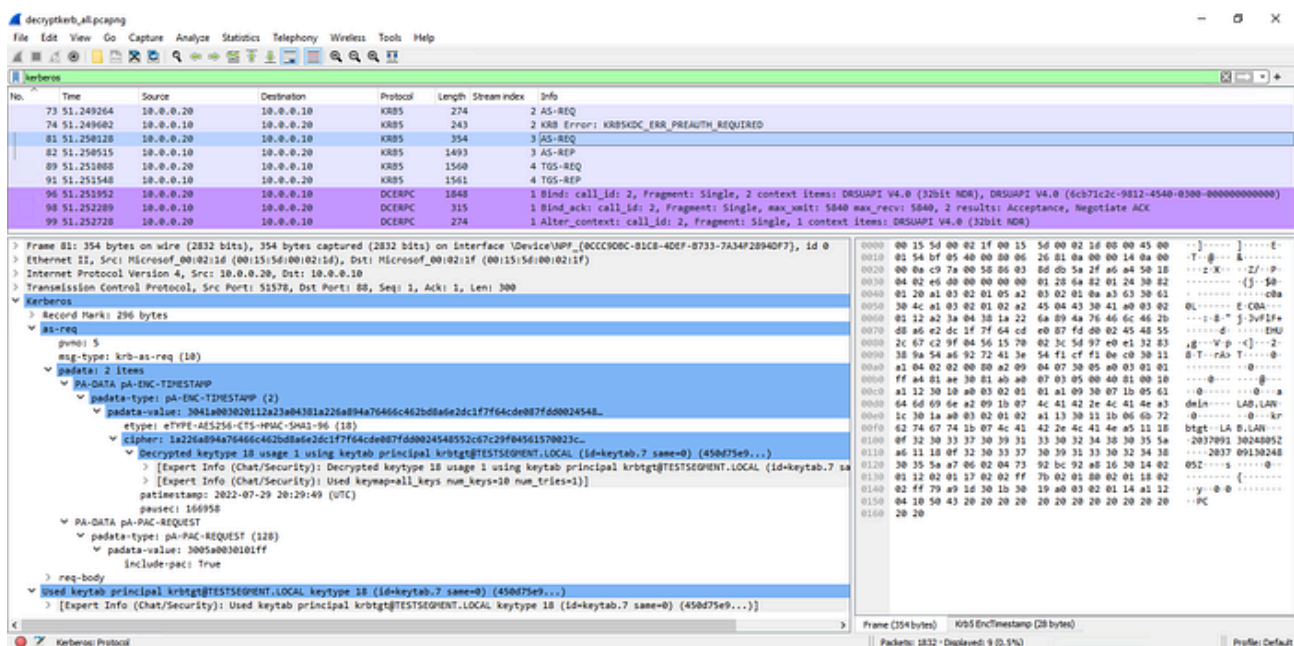
Check "Try to decrypt Kerberos blobs" and Browse to the location of the keytab file you just generated.



Decrypt Kerberos

Now you can try opening some Kerberos exchanges. Everything that is properly decrypted will be highlighted in light blue. Here are a couple examples:

Press enter or click to view image in full size



AS-REQ with the decrypted timestamp

Press enter or click to view image in full size

Wireshark capture of an AS-REP message. The left pane shows the packet list with packet 82 selected. The middle pane shows the packet details for 'AuthorizationData item', including 'PAC_LOGON_INFO' and 'PAC_HEADER'. The right pane shows the raw packet data in hexadecimal and ASCII. The status bar at the bottom indicates 'Frame 1403 (bytes)' and 'Krb5 Ticket (393 bytes)'.

AS-REP with the decrypted PAC (containing the user's privileges, see [MS-PAC](#))

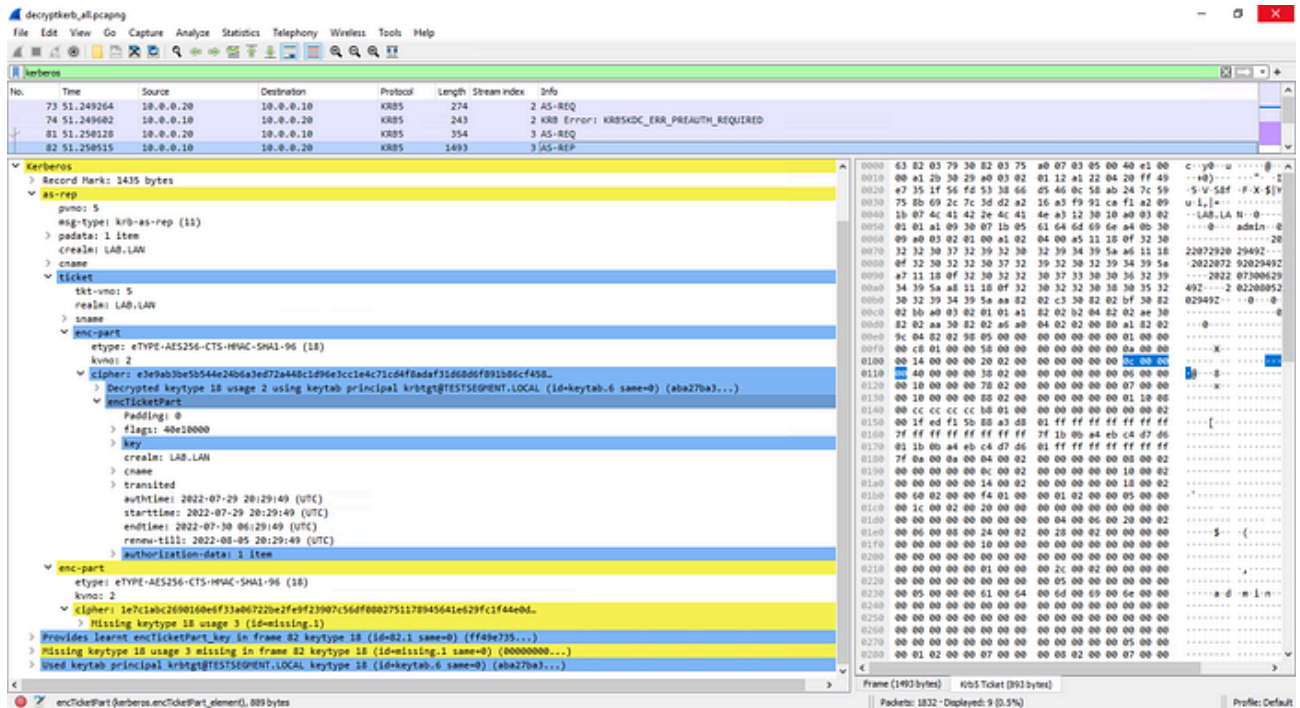
Press enter or click to view image in full size

Wireshark capture of a TGS-REP message. The left pane shows the packet list with packet 99 selected. The middle pane shows the packet details for 'TGS-REP', including 'TGS-REP-Part' and 'TGS-REP-Part'. The right pane shows the raw packet data in hexadecimal and ASCII. The status bar at the bottom indicates 'Frame 1561 (bytes)' and 'Krb5 Ticket (3052 bytes)'.

TGS-REP with its two parts, including the service ticket, both containing the same session key

⚠ If you notice parts highlighted in yellow it means that the decryption failed. Perhaps the corresponding key is missing in the keytab, or its value for the selected algorithm was not provided (check the “etype” field to see which algorithm is used). For example:

Press enter or click to view image in full size



🤖 Surprise test about Kerberos theory: can you guess whose key I provided here, and whose key is missing?

Answer: We observe that Wireshark can decrypt the first part which is the TGT encrypted with the KDC key, but it cannot decrypt the second part which is encrypted with the client's key. Therefore, here the keytab only contains the krbtgt key.

Decrypt DCE/RPC, LDAP...

Do you remember how this all began? I wanted to decrypt DCERPC payloads, not the Kerberos protocol itself!

And... it works too! 🌟

Quick reminder first, the same color rule applies: blue means that decryption is ok, and yellow means errors. If you see some yellow during the authentication phase of the protocol (here the Bind step) the rest will certainly cannot be decrypted:

Press enter or click to view image in full size

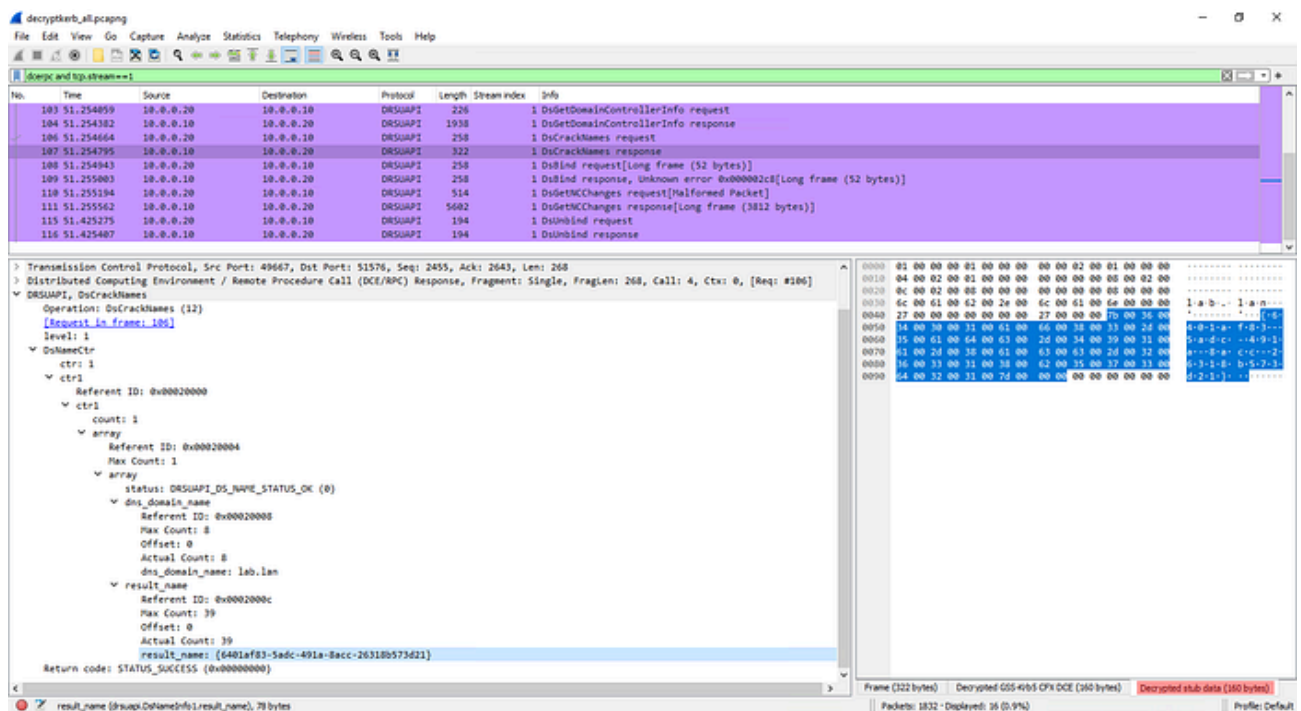
The image shows a Wireshark packet capture of a Kerberos authentication process. The top pane displays a list of packets, with packet 11 selected. The middle pane shows the details of packet 11, which is a 'Kerberos' message. The bottom pane shows the raw packet data in hexadecimal and ASCII. The details pane is expanded to show the 'Authenticating Party' field, which contains the text 'Kerberos'. The details pane also shows the 'Authenticating Party' field, which contains the text 'Kerberos'. The details pane also shows the 'Authenticating Party' field, which contains the text 'Kerberos'.

Here are some examples where it works, notice how the “encrypted stub data” is now replaced with “decrypted stub data” 🏆

Press enter or click to view image in full size

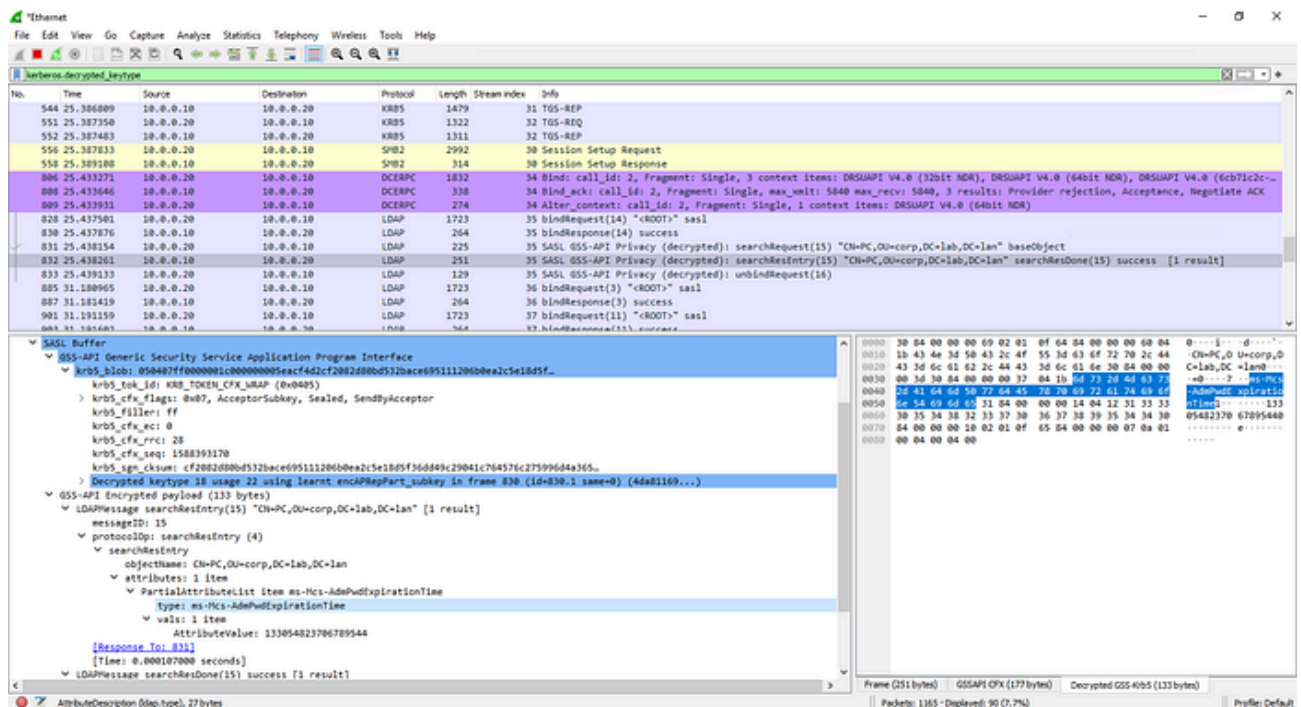
The image shows a Wireshark packet capture of a Kerberos authentication process. The top pane displays a list of packets, with packet 107 selected. The middle pane shows the details of packet 107, which is a 'Kerberos' message. The bottom pane shows the raw packet data in hexadecimal and ASCII. The details pane is expanded to show the 'Authenticating Party' field, which contains the text 'Kerberos'. The details pane also shows the 'Authenticating Party' field, which contains the text 'Kerberos'. The details pane also shows the 'Authenticating Party' field, which contains the text 'Kerberos'.

Press enter or click to view image in full size



It also works with other protocols, like LDAP:

Press enter or click to view image in full size



workstation checking if its LAPS password is expired, and thus due for renewal

Tip to refresh the keytab

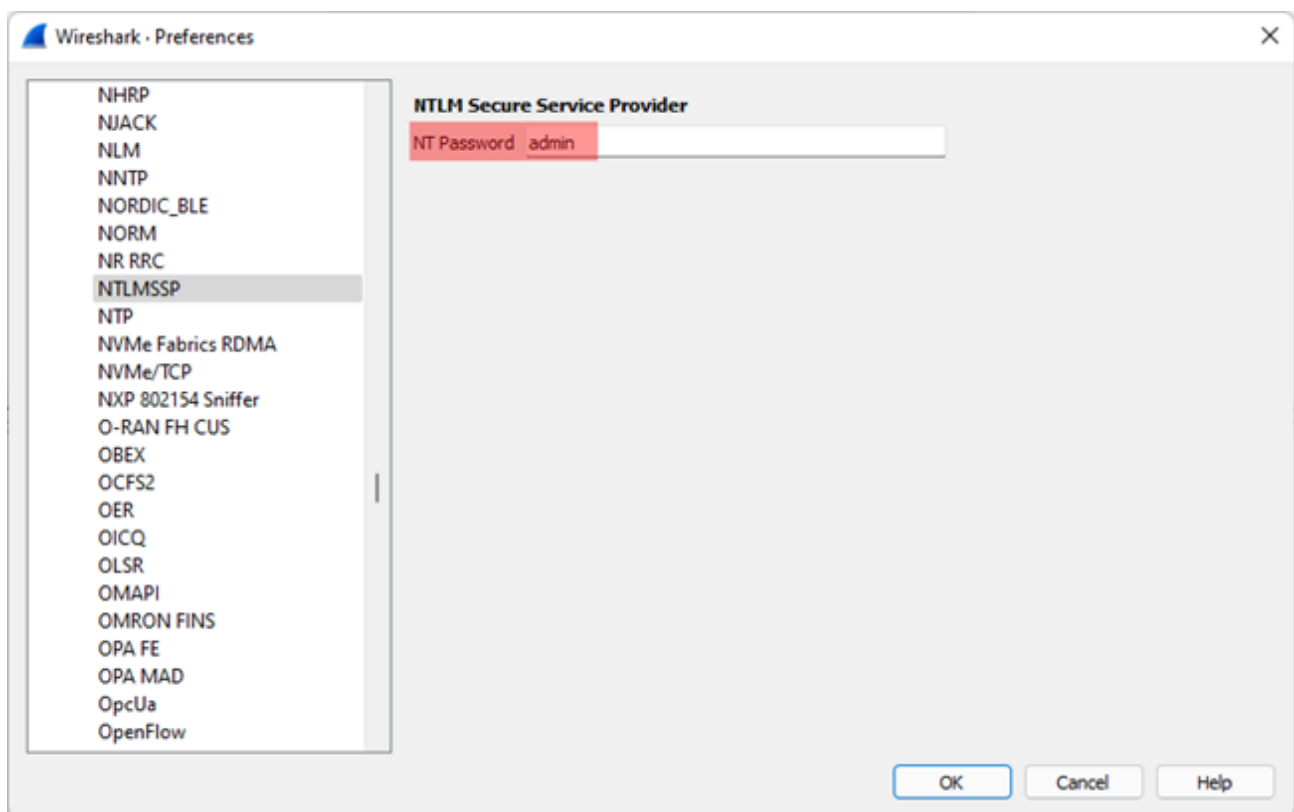
A modified keytab file does not take effect immediately in Wireshark. Either you have to open the Preferences, disable Kerberos decryption, confirm, then re-open it to re-enable it,

which is slow and annoying... Or the fastest I've found is to save the capture, close Wireshark and re-open the capture file.

NTLM decryption

What about NTLM? Can we do the same decryption if NTLMSSP authentication is used? The answer is yes! 😊

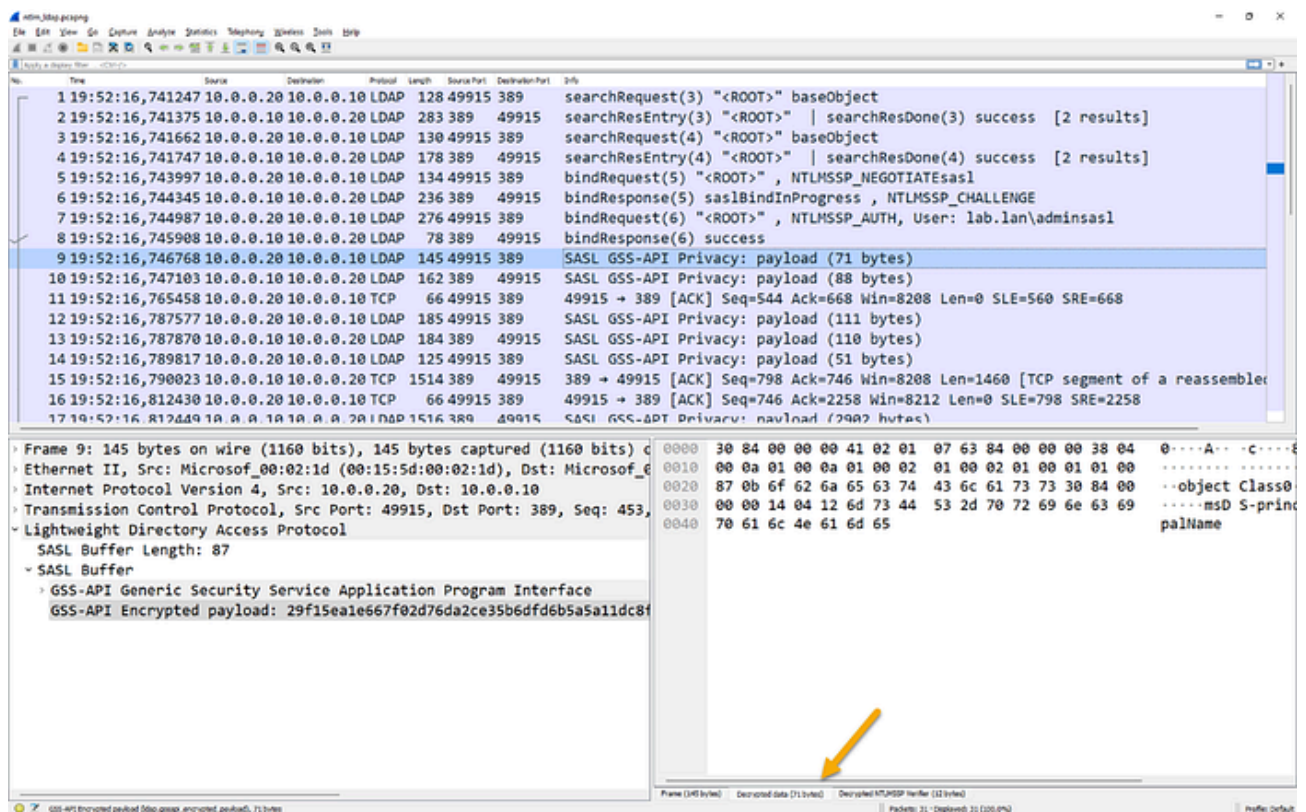
In the Preferences, scroll to the “NTLMSSP” protocol, and type the cleartext password in the “NT Password” field. This is described in the [Wireshark NTLMSSP wiki page](#) where I have added some examples.



Some limitations contrary to Kerberos: you need the cleartext password and it must be ASCII only (this limitation is mentioned in the source code) so it is not applicable to machine account passwords, and you can only provide one at a time, contrary to the keytab which can hold keys for several users.

👁️ Do not forget to click the “Decrypted data” tab at the bottom:

Press enter or click to view image in full size



Update: actually, it is possible to decrypt using NTLM hash(es)! This feature is not documented, and not possible through the UI, but by looking at the code we can see that it is indeed possible as described in this CTF writeup: [Insomni'Hack Teaser 2023 — Autopsy](#). How to provide the NT hash(es)? Using a keytab too! It's a bit confusing to use a Kerberos option to decrypt NTLMSSP but it works. If you remember earlier, I said that the RC4 key to put in a keytab is identical to the NT hash. So, you have to create a keytab entry, as explained in [Insomni'Hack Teaser 2023 — Autopsy](#), using the RC4-HMAC type (etype 23) and with the NT hash. Enable it in the Wireshark KRB5 options, same as before, and your NTLM encrypted traffic will be in clear-text if the hash is correct.

Conclusion

I hope these tips will help you in your journey to examine “encrypted stub data” payloads using Wireshark. This is something that we often do at Tenable when doing research on Active Directory, and I hope it will benefit you too!

Protocols become increasingly encrypted by default, which is a very good thing... Therefore, packet capture analysis, without decryption capabilities, will become less and less useful, and I'm thankful to see those tools including such features. Do you know other protocols that Wireshark can decrypt? Or perhaps with other tools?